It's important to understand that this doesn't transform the data of the variable. It simply reassigns the variable to a different value.

One thing that trips up many new Elixir programmers is that you cannot have function calls inside of the pattern.

```
iex> 1 + 1 = 2
CompileError: cannot invoke remote function :erlang.+/2 inside a match
```

```
iex>
defmodule Local do
  def call do
    test() = 1
  end

  defp test do
    1
  end
end
```

```
** (CompileError): cannot find or invoke local test/0 inside match.
   Only macros can be invoked in a match and they must be defined
   before their invocation. Called as: test()
```

This is a useful error message. It tells us about our coding error and also lets us know that some functions (macros) can be invoked in a match clause.

It's not common to write macro-based match functions yourself, but you'll frequently use ones provided by Elixir. Besides lists and maps—which we will cover next—string concatenation is commonly used. Here's an example that uses string concatenation (<>) in a match clause:

```
iex> "store:" <> data_command = "store:Widget:process"
"store:Widget:process"
```

```
iex> data_command
"Widget:process"
```

The <> appears on the left side of the = symbol, and a variable is used where a string part would be. Elixir pattern-matches the string and extracts the relevant text into the data_command variable.

This is a powerful way to split apart text without calling String.split/2. But it's not without its limitations. The variable must always be the last part of the concatenation. You can do this:

```
iex> "text" <> ":" <> number = "text:7"
iex> number
"7"
```